

PROJECT REPORT

Implement Client Script and UI Policy (Incident)

Team ID	SWTID-2026-8511
Date	1 October 2026
Team Members	[Add team member names and register numbers]
Department	Computer Science and Business Systems, Jaya Engineering College
Platform	ServiceNow (Incident Management)

TABLE OF CONTENTS

1. INTRODUCTION	3
1.1 Project Overview	3
1.2 Purpose	3
2. IDEATION PHASE	3
2.1 Problem Statement	3
2.2 Empathy Map Canvas	3
2.3 Brainstorming	4
3. REQUIREMENT ANALYSIS	5
3.1 Customer Journey Map	5
3.2 Solution Requirement	6
3.3 Data Flow Diagram	7
3.4 Technology Stack	8
4. PROJECT DESIGN	9
4.1 Problem Solution Fit	9
4.2 Proposed Solution	9
4.3 Solution Architecture	10
5. PROJECT PLANNING & SCHEDULING	10
5.1 Project Planning	10
6. FUNCTIONAL AND PERFORMANCE TESTING	11
6.1 Performance Testing	11
7. RESULTS	12
7.1 Output Screenshots	12
8. ADVANTAGES & DISADVANTAGES	12
Advantages	12
Disadvantages	12
9. CONCLUSION	12
10. FUTURE SCOPE	12
11. APPENDIX	13
Source Code	13
Dataset Link	13

1. INTRODUCTION

1.1 Project Overview

In ServiceNow, service desk agents use the Incident form to log and update the issues that callers report. When the form has no rules, agents can miss mandatory or conditional fields, see fields that do not apply, and save wrong or incomplete data. This is usually found only after saving, which leads to rework and reopened incidents.

This project implements **UI Policies** and **Client Scripts** on the Incident table. The UI Policy shows or hides fields and makes fields mandatory or read-only based on conditions such as Category, Priority and State. The Client Scripts react to field changes (onChange) and validate the details when the agent submits the form (onSubmit). The agent therefore gets guidance while filling the form, not after saving it.

1.2 Purpose

The purpose of the project is to:

- Help agents log complete and correct incidents quickly.
- Show only the fields that are relevant and require the fields that are needed.
- Give clear, instant messages when details are missing or a value needs attention.
- Reduce rework, reopened incidents and slow resolution.
- Give beginners practical experience with Client Scripts and UI Policies in ServiceNow.

2. IDEATION PHASE

2.1 Problem Statement

The problem is described from the customer's point of view in the format: I am, I'm trying to, but, because, which makes me feel.

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	A service desk agent who creates and updates incidents in ServiceNow daily	Fill in the Incident form quickly and with correct details	Mandatory or conditional fields (e.g., Priority, Assignment group, Resolution notes) get missed and wrong data is saved	The Incident form has no Client Script or UI Policy to validate input and show or hide fields	Frustrated and unsure about the data quality
PS-2	A ServiceNow administrator who maintains the Incident form for the support team	Make the form guide users automatically (mandatory fields, read-only fields, alerts) without server-side delays	Users still skip required details and see fields that are not relevant, so incidents are incomplete and slow to resolve	Form behavior is not controlled on the client side, so rules are not applied as the user fills the form	Concerned and overloaded with cleanup and follow-up work

2.2 Empathy Map Canvas

The empathy map captures what the service desk agent thinks and feels, hears, sees, says and does, along with the pain and gain. The main pain is missed mandatory fields and wrong data saved. The main gain is a form that validates input instantly and shows only relevant fields.

Empathy Map: Incident Form with Client Script & UI Policy

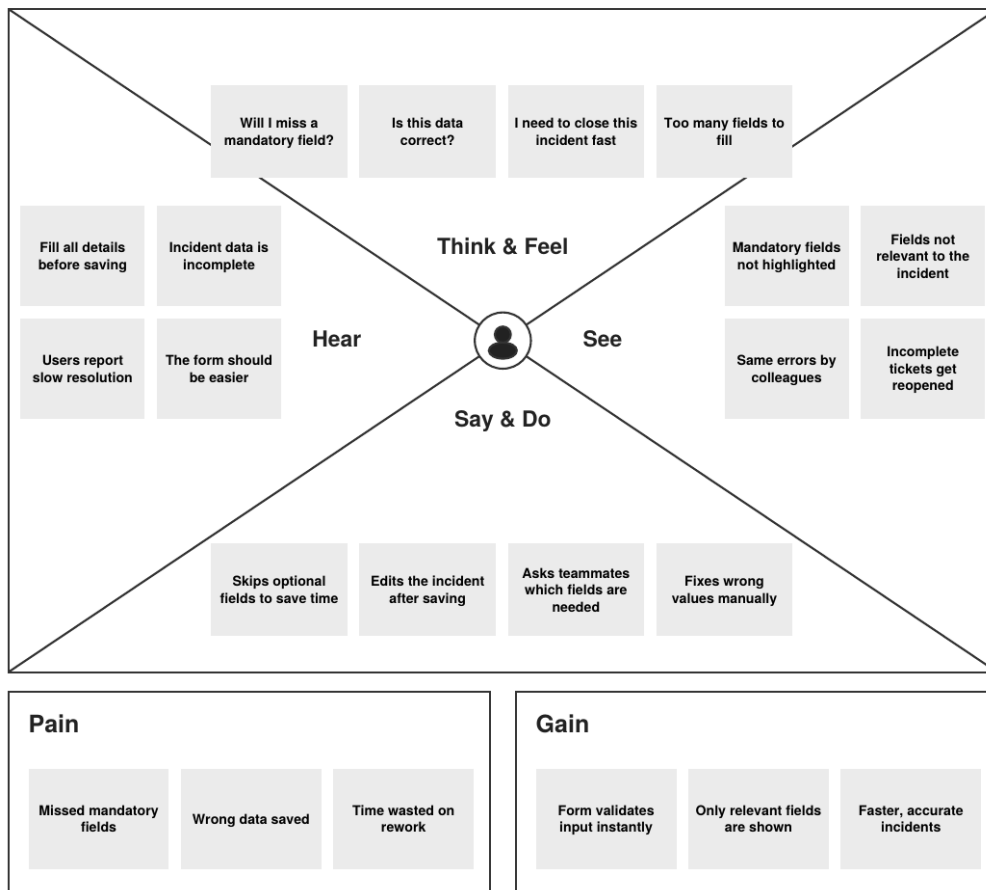


Figure 2.1: Empathy map for the service desk agent

2.3 Brainstorming

The team listed possible ways to solve the problem and chose the ones that give feedback while the agent is filling the form.

Idea	Pros	Cons	Decision
Server-side Business Rules for validation	Cannot be bypassed	Error appears only after saving	Future scope
UI Policy to show/hide fields and set mandatory or read-only	No code needed, instant effect	Handles simple conditions only	Selected
Client Script (onChange) to alert or guide on field change	Instant feedback, flexible logic	Needs scripting and care for speed	Selected
Client Script (onSubmit) to validate before saving	Stops incomplete incidents early	Can be bypassed outside the form	Selected
Training document for agents	Easy to prepare	Does not stop mistakes	Not selected

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The journey map follows the agent through five stages: Entice, Enter, Engage, Exit and Extend. The Engage and Exit stages are where the UI Policy and Client Scripts help most.

Customer Journey Map

Implement Client Script and UI Policy (Incident) | Service desk agent logging an incident in ServiceNow

Date: 1 October 2026
Team ID: SWTID-2026-8511
Project: Implement Client Script and UI Policy (Incident)

SCENARIO Creating and saving an incident on the Incident form	Entice How does the agent first become aware of the need?	Enter What does the agent experience as they begin?	Engage In the core moments, what happens?	Exit What happens as the process finishes?	Extend What happens after the incident is saved?
Steps What does the agent typically do?	A caller reports an issue. The agent clicks New in the Incident module.	The Incident form opens. The agent enters Caller, Short description and Category.	The agent sets Category, Priority and State. The form reacts through UI Policy and Client Script.	The agent clicks Submit or Save. The form checks the data first.	The incident is assigned, worked on, resolved, and used in reports.
Interactions What people, places and things are involved?	People: caller, team lead. Things: phone, email, Service Portal.	Place: ServiceNow Incident form (Create New) in the browser.	Form fields, UI Policy actions (mandatory, visible, read-only) and onChange Client Script messages.	Submit and Save buttons, onSubmit Client Script messages.	Resolver group, notification email to the caller, incident list and reports.
Goals & motivations At each step, what is the agent's main goal?	Help me log the issue quickly.	Help me know which fields I must fill.	Help me avoid missing or wrong data.	Help me save the incident without errors.	Help me see the incident resolved with clean data.
Positive moments Which steps feel easy, useful or satisfying?	The caller gives a clear issue to log.	The form layout is familiar.	Needed fields turn mandatory and unrelated fields hide automatically.	A clear message shows exactly what is missing before saving.	Complete data speeds up assignment and resolution.
Negative moments Which steps feel frustrating, confusing or slow?	Callers give unclear or partial details.	There are many fields and it is unclear which ones matter.	Without rules, required fields are skipped and wrong values are entered.	Errors are found only after saving, so the agent redoes the work.	Incomplete incidents are reassigned or reopened.
Areas of opportunity How might we make each step better?	Show short help text on the form about what details to collect.	Use a UI Policy to hide fields that are not needed.	Use UI Policy for mandatory and read-only fields by Priority or State, and an onChange Client Script to guide input.	Use an onSubmit Client Script to validate and show a clear message.	Track fewer reopened incidents once the rules are in place.

Figure 3.1: Customer journey map

3.2 Solution Requirement

Functional requirements:

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	Incident Creation	Open the Incident form; enter caller, short description and category; auto-fill default values on form load
FR-2	Dynamic Form Behaviour (UI Policy)	Show or hide fields by Category or State; make fields mandatory by Priority or State; make fields read-only when the incident is Closed
FR-3	Form Validation (Client Script)	onLoad: set defaults and show form messages; onChange: alert or auto-fill when Priority, Impact or Urgency change; onSubmit: validate details and stop the submit with a clear message
FR-4	Configuration and Administration	Create, activate and deactivate UI Policies and Client Scripts on the Incident table; set conditions and execution order
FR-5	Testing and Verification	Test every rule with sample incidents; confirm fields, messages and validations behave as expected

Non-functional requirements:

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	The form is simple to use. Messages are short and clear, and only relevant fields are shown.
NFR-2	Security	Only users with the administrator role can create or change UI Policies and Client Scripts. Server-side access controls still apply.
NFR-3	Reliability	Rules apply the same way every time, and the UI Policies and Client Scripts do not conflict.
NFR-4	Performance	Rules run in the browser without a noticeable delay. Client Scripts avoid heavy server calls.
NFR-5	Availability	Rules are available whenever the ServiceNow instance is available. A rule can be deactivated quickly.
NFR-6	Scalability	New rules and fields can be added and the approach reused on other tables. Many agents can use the form at the same time.

3.3 Data Flow Diagram

The Level 0 data flow diagram shows how incident details move between the caller, the service desk agent, the UI Policy and Client Script processes, the rules store (D1) and the Incident table (D2).

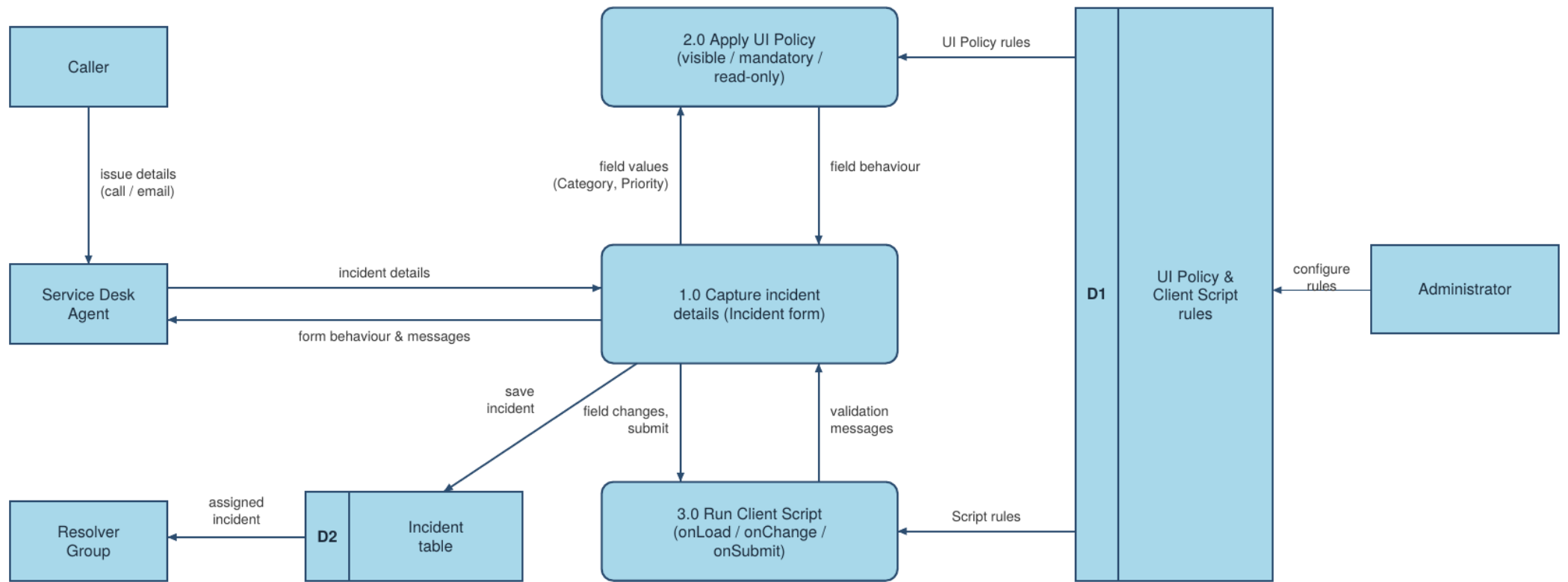


Figure 3.2: Data flow diagram (Level 0)

3.4 Technology Stack

The solution runs on a cloud-hosted ServiceNow instance. Rules run in the browser and the server-side logic and database are handled by the platform.

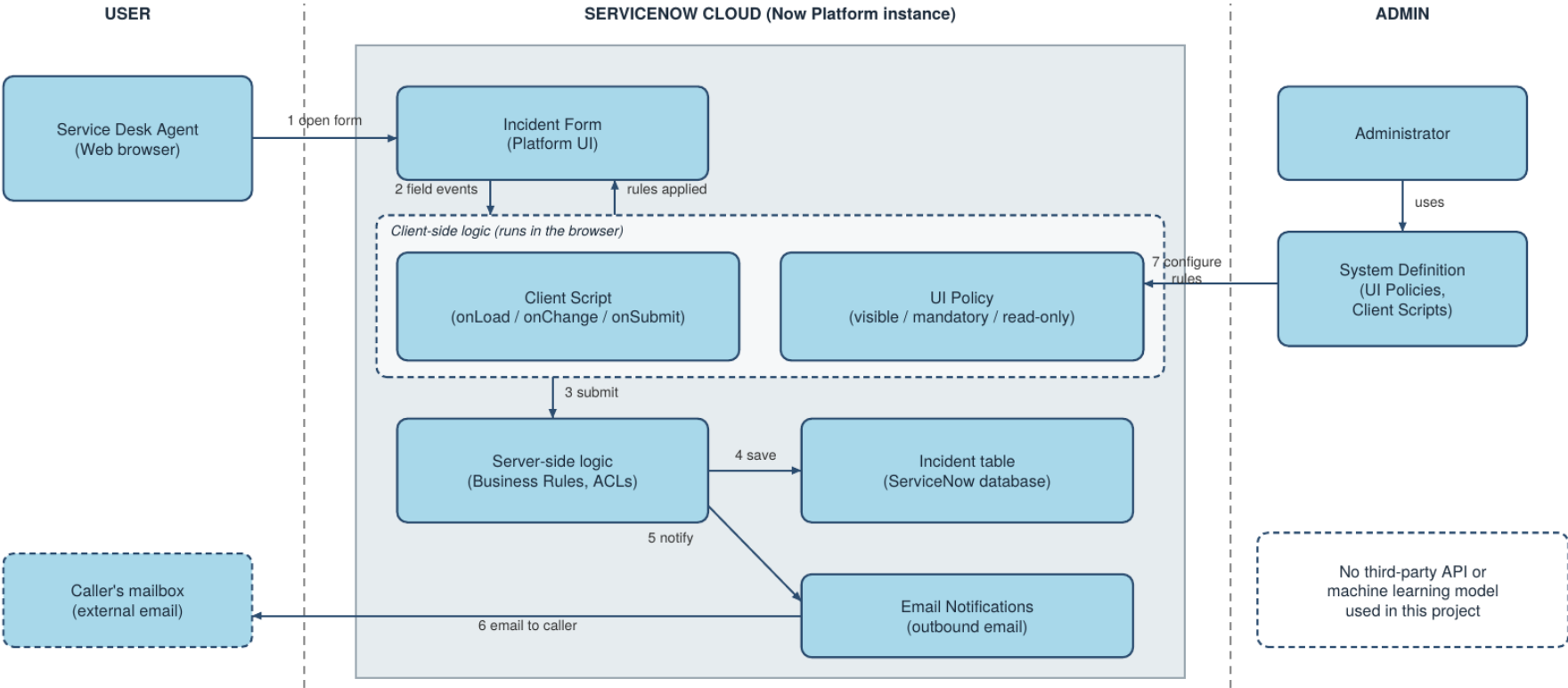


Figure 3.3: Technical architecture

Component	Technology
User Interface	ServiceNow Platform UI (Incident form), HTML, CSS, JavaScript
Application Logic-1	Client Script (JavaScript, g_form API)
Application Logic-2	UI Policy (conditions and UI Policy Actions)
Application Logic-3	Business Rules and ACLs (server-side JavaScript)
Database	ServiceNow Incident table (incident)
External interface	ServiceNow outbound email (SMTP)
Infrastructure	ServiceNow Personal Developer Instance (PDI), cloud-hosted
External API / ML model	Not used

4. PROJECT DESIGN

4.1 Problem Solution Fit

The problem-solution fit checks that the solution actually solves the customer's problem. The canvas below covers the customer segment, constraints, root cause, behaviour, triggers, emotions and the solution.

Define CS, fit into CC	1. CUSTOMER SEGMENT(S) CS - Service desk agents who log and update incidents in ServiceNow every day - Administrators who maintain the Incident form - Resolver groups who receive the incidents	6. CUSTOMER CONSTRAINTS CC - Time pressure to log incidents fast - Beginner agents may not know which fields matter - Limited admin time for cleanup - Must work inside the standard ServiceNow UI with no extra cost or plugins	5. AVAILABLE SOLUTIONS AS - Team lead checks incidents by hand (slow, not always done) - Fixing incidents after they are saved (rework) - Server-side Business Rules only (error shows only after saving)	Explore AS, differentiate
	2. JOBS-TO-BE-DONE / PROBLEMS J&P - Log complete and correct incidents quickly - Know which fields are required - Avoid wrong values - Problems: missed mandatory fields, irrelevant fields on the form, errors found after saving	9. PROBLEM ROOT CAUSE RC - The Incident form has no client-side rules - Validation happens only after saving, or not at all - Fields that are required depend on Priority, State or Category, but the form does not check them	7. BEHAVIOUR BE - Skips optional fields to save time - Edits the incident after saving - Asks teammates which fields are needed - Fixes wrong values manually - Incidents get reassigned or reopened	
Identify strong TR & EM	3. TRIGGERS TR - A caller reports an issue and a new incident must be logged - Team lead points out incomplete incidents - Reopened incidents keep increasing	10. YOUR SOLUTION SL - Add a UI Policy on the Incident form to show or hide fields and make fields mandatory or read-only by Category, Priority and State - Add Client Scripts (onChange, onSubmit) to alert the agent and validate details before saving - Test the rules with sample incidents - Result: complete, correct incidents with instant feedback	8.1 ONLINE CH - ServiceNow Incident form and Incident list in the browser - Email and Service Portal where callers report issues	Extract online & offline CH or BE
	4. EMOTIONS: BEFORE / AFTER EM - Before: frustrated, rushed, unsure about data quality - After: confident, in control, faster		8.2 OFFLINE CH - Phone calls from callers - Talks with team leads and teammates; use these to train agents on the new rules	

Figure 4.1: Problem-solution fit canvas

4.2 Proposed Solution

S.No.	Parameter	Description
1.	Problem Statement	Agents miss mandatory or conditional fields and save wrong or incomplete data because the Incident form has no client-side rules.
2.	Idea / Solution description	A UI Policy shows or hides fields and makes them mandatory or read-only by Category, Priority and State. Client Scripts (onChange, onSubmit) alert the agent and validate the details before saving.
3.	Novelty / Uniqueness	Rules run in the browser, so feedback comes while filling the form. UI Policy handles simple behaviour without code, and Client Scripts handle only the checks that need logic. No extra plugin or cost is needed.
4.	Social Impact / Customer Satisfaction	Less time on corrections, more confidence in the data, faster resolution for callers, and beginners learn which fields matter from the form.
5.	Business Model (Revenue Model)	A learning project with no direct revenue. The value is saved agent time and fewer reopened incidents. It could be offered as a configuration service.

S.No.	Parameter	Description
6.	Scalability of the Solution	The approach can be reused on other tables (Problem, Change) and extended with new fields and conditions. It runs on the ServiceNow platform for many concurrent agents.

4.3 Solution Architecture

The agent opens the Incident form, the UI Policy and Client Script apply while the form is filled, the server-side logic validates on save, and the incident is stored and assigned to the resolver group.

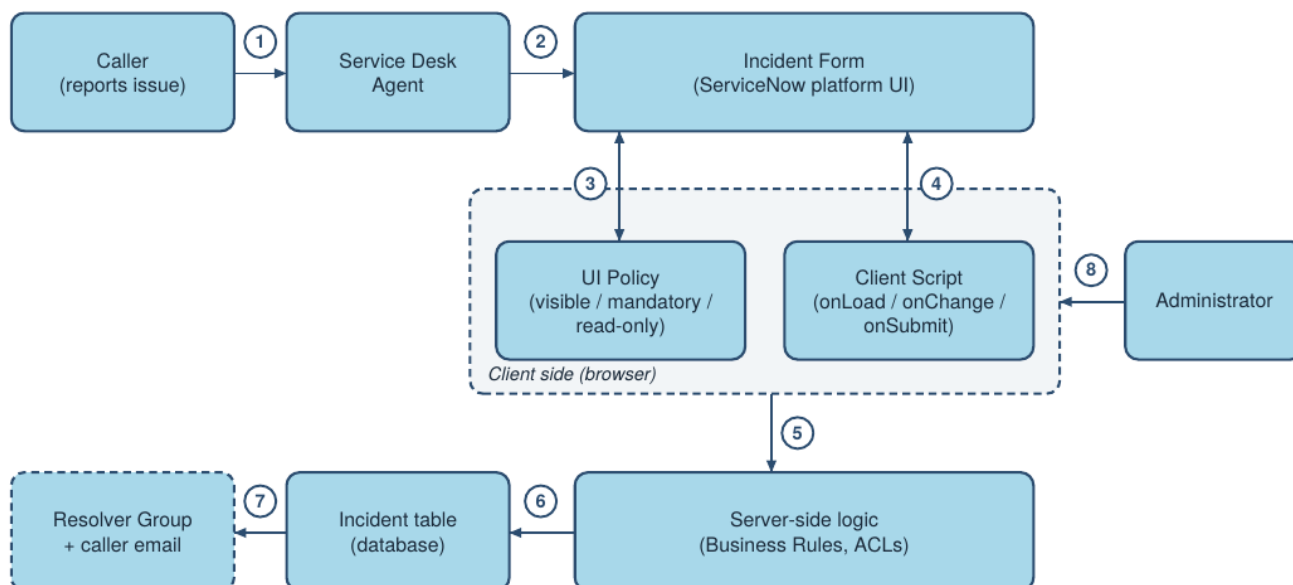


Figure 4.2: Solution architecture

1. The caller reports an issue and the service desk agent starts a new incident.
2. The agent opens the Incident form in the ServiceNow platform UI.
3. The UI Policy shows or hides fields and makes them mandatory or read-only.
4. The Client Script reacts to field changes (onChange) and validates on submit (onSubmit).
5. The validated incident goes to the server-side logic (Business Rules, ACLs).
6. The incident is saved in the Incident table.
7. The incident is assigned to the resolver group and the caller is notified by email.
8. The administrator creates and maintains the rules.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The work is split into three 6-day sprints with 10 user stories and 40 story points in total.

Sprint	Story No.	User Story / Task	Points	Priority
Sprint-1	USN-1	Open the Incident form and enter caller, short description and category	2	High
Sprint-1	USN-2	See only the fields that are relevant to the incident	3	High
Sprint-1	USN-3	Required fields become mandatory automatically	3	High
Sprint-1	USN-7	Administrator creates and activates a UI Policy on the Incident table	3	High
Sprint-1	USN-8	Administrator creates a Client Script on the Incident table	3	High
Sprint-2	USN-4	Fields become read-only once an incident is closed	3	Medium
Sprint-2	USN-5	Alert when Priority, Impact or Urgency changes (onChange)	5	Medium
Sprint-2	USN-6	Clear message if details are missing on submit (onSubmit)	5	High
Sprint-3	USN-9	Administrator tests the rules with a sample incident	8	Medium
Sprint-3	USN-10	Resolver receives incidents with complete and correct details	5	Medium

Sprint	Total Story Points	Duration	Start Date	End Date (Planned)	Points Completed	Release Date (Actual)
Sprint-1	14	6 Days	5 Oct 2026	10 Oct 2026		
Sprint-2	13	6 Days	12 Oct 2026	17 Oct 2026		
Sprint-3	13	6 Days	19 Oct 2026	24 Oct 2026		

Average velocity is $40 / 3 = 13.3$ story points per sprint. Average velocity per day is $13.3 / 6 = 2.2$ story points per day. The planned burndown chart is shown below; actual points remaining are added at the end of each sprint.

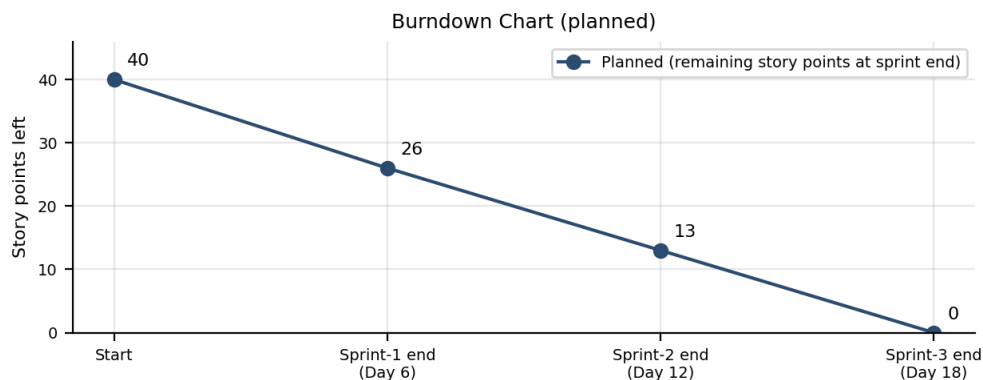


Figure 5.1: Planned burndown chart

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Each rule is tested on the Incident form with sample incidents. Record the actual result and the status (Pass/Fail) after running each test.

Test	Scenario	Expected Result	Actual Result	Status
TC-1	Open a new incident	Form loads with the default fields and values		
TC-2	Set State to Resolved and leave Resolution notes empty	Save is blocked; Resolution notes is mandatory (UI Policy)		
TC-3	Change Category	Relevant fields are shown and others are hidden (UI Policy)		
TC-4	Set State to Closed	Fields become read-only (UI Policy)		
TC-5	Change Impact to High	An alert message appears immediately (onChange)		
TC-6	Submit with Short description empty	Submit is stopped and a message names the missing field (onSubmit)		
TC-7	Submit a complete incident	Incident is saved successfully		
TC-8	Open the Client Script list as a non-administrator	Creating or editing scripts is not allowed		

Performance check: measure the form load time with the rules active and with the rules deactivated, using the browser developer tools, and enter the readings below. The rules should add no noticeable delay.

Measurement	Reading	Notes
Form load time, rules active (seconds)		Browser:
Form load time, rules deactivated (seconds)		
Time to show onChange alert (seconds)		

7. RESULTS

7.1 Output Screenshots

Output screenshots are not included in this report.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Agents get instant feedback while filling the form.
- Fewer missed fields and less wrong data, so less rework.
- UI Policy needs no code for simple field behaviour.
- Uses standard ServiceNow features with no extra cost or plugin.
- Rules can be reused on other tables and forms.

Disadvantages

- Client-side rules can be bypassed by imports and integrations, so server-side checks are still needed.
- Too many Client Scripts can slow the form.
- Rules depend on field names and choice values, so they need maintenance when the form changes.
- Rules apply only on the form, not to data that arrives through the API.

9. CONCLUSION

The project shows that UI Policies and Client Scripts make the ServiceNow Incident form easier and safer to use. The UI Policy handles field visibility, mandatory and read-only behaviour, and the Client Scripts add alerts and validation that need logic. Together they help agents log complete and correct incidents, reduce rework and speed up resolution. Server-side rules should still back them up where data must never be wrong.

10. FUTURE SCOPE

- Add Data Policies and Business Rules so the rules also apply to imports and integrations.
- Apply the same approach to the Problem and Change forms.
- Add Client Scripts to the Service Portal and catalog forms so callers also get guidance.
- Build a dashboard that tracks reopened incidents before and after the rules.
- Use Predictive Intelligence to suggest Category and Assignment group.

11. APPENDIX

Source Code

Sample scripts for the Incident table. Replace them with your final scripts before submission.

Client Script (onSubmit): validate Resolution notes

```
function onSubmit() {  
    var state = g_form.getValue('state');  
    var notes = g_form.getValue('close_notes');  
    // 6 = Resolved  
    if (state == '6' && !notes) {  
        g_form.addErrorMessage('Please enter Resolution notes before resolving.');        return false;    // stops the submit  
    }  
}
```

Client Script (onChange on field Impact): alert for high impact

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {  
    if (isLoading || newValue === '') {  
        return;  
    }  
    // 1 = High  
    if (newValue == '1') {  
        g_form.showFieldMsg('impact', 'High impact selected. Please confirm the business impact.', 'info');  
    }  
}
```

UI Policy: make Resolution notes mandatory when Resolved

Table: Incident [incident]
Condition: State is Resolved
Actions: Field name = Resolution notes (close_notes)
Mandatory = True, Visible = True

UI Policy: make fields read-only when Closed

Table: Incident [incident]
Condition: State is Closed
Actions: Field name = Short description, Read only = True
Field name = Description, Read only = True

Dataset Link

Not applicable. The project does not use an external dataset.